TECHNICAL DOCUMENTATION

PERK LOOP

Complete engineering reference for the Perk Loop loyalty platform - frontend, backend, data model, and the security architecture that protects every point. Includes the production deployment checklist and free-tier service plan for the first 100 customers.

Perk Loop Engineering

Version 1.0 · October 2026 · Noida, India

EVERY VISIT COUNTS

Table of Contents

1. Product Overview and Build Status 2

2. System Architecture 3

2.1 Single-Page Shell and Hash Routing . . . 3

2.2 Layer Diagram . . . 3

2.3 Request Lifecycle . . . 3

2.4 Session Model . . . 4

3. Frontend Implementation 4

3.1 Route Map . . . 5

3.2 Design System Highlights . . . 5

4. Backend Implementation 6

4.1 Lib Modules . . . 7

4.2 Loyalty Engine Functions . . . 7

5. Data Model 8

6. Feature Walkthroughs 9

6.1 Signup and OTP Login . . . 9

6.2 Static Check-in QR and Stamping . . . 10

6.3 One-Time Amount QR (Purchase Points) . . . 10

6.4 Redemption Approval Loop . . . 11

6.5 Feedback and Notifications . . . 11

7. Security Architecture 11

7.1 Threat Model . . . 11

7.2 Why Points Cannot Be Faked . . . 12

7.3 Already Implemented . . . 12

7.4 Production Hardening Checklist . . . 13

8. Production Deployment Checklist 14

8.1 Environment Variables . . . 14

8.2 Steps . . . 14

8.3 Post-Deploy Smoke Checklist . . . 14

9. Free-Tier Service Selection (Pre-Launch to First 100 Customers) 15

10. Optional Later (Phase 2 and Beyond) 16

10.1 What Ships Today . . . 17

1. Product Overview and Build Status

Perk Loop is an India-first loyalty and rewards platform for local shops, launching in Noida and Greater Noida. Customers earn stamps and points at the places they already visit - cafes, salons, bakeries, street food stalls - through QR codes at the counter, with no app download. Merchants run their entire program from a browser dashboard: program builder, QR studio, transaction log, redemption approvals and honest analytics. The product promise is trust: every balance is per-business, every point movement is recorded in an append-only ledger, and nothing is added to a balance without a real, server-confirmed visit or purchase.

The platform ships as one Next.js 16 application with three user-facing surfaces inside a single-page shell, backed by 27 JSON API route handlers and a 15-model Prisma database.

27	15	31/31	0
API endpoints	database models	E2E checks passing	console errors

Phase	Deliverable	Status
0 - Foundation	Design system (DESIGN.md), fonts, CSS tokens, hash router, session store	Done, verified
1 - Database	Final Prisma schema (15 models), seed = labeled Demo Cafe only	Done, verified
2 - Auth	OTP request/verify APIs, rate limits, JWT cookie session, /api/me	Done, verified
3 - Loyalty engine	Program rules, earn/stamp/redeem APIs, atomic issuance, ledger, audit	Done, verified
4 - Customer app	Auth screens, wallet, discover, business wallet, scan confirm, feedback	Done, browser-verified
5 - Merchant app	Onboarding, program builder, QR studio, transactions, redemptions, analytics, settings	Done, browser-verified
6 - Landing page	Hero, how-it-works, categories rail, for-business, previews, FAQ, legal pages	Done, browser-verified
7 - Hardening + E2E	Replay/negative/isolation tests, lint, responsive audit, data cleanup	31/31 pass

Table 1 - Build phases and completion status.

2. System Architecture

2.1 Single-Page Shell and Hash Routing

The entire user experience lives in `src/app/page.tsx` as a client-side single-page application driven by hash routing (for example #/app, #/merchant/qr, #/business/id). This decision comes from a deployment constraint of the development sandbox, which exposes only the root path, but it also suits the product: the customer wallet must open instantly from a scanned QR code, with no full page loads between screens. A tiny custom router (`src/lib/hash-router.ts`, 40 lines) parses the hash into a route object, and a zustand store holds the hydrated session. The same architecture runs unchanged on Vercel in production.

2.2 Layer Diagram

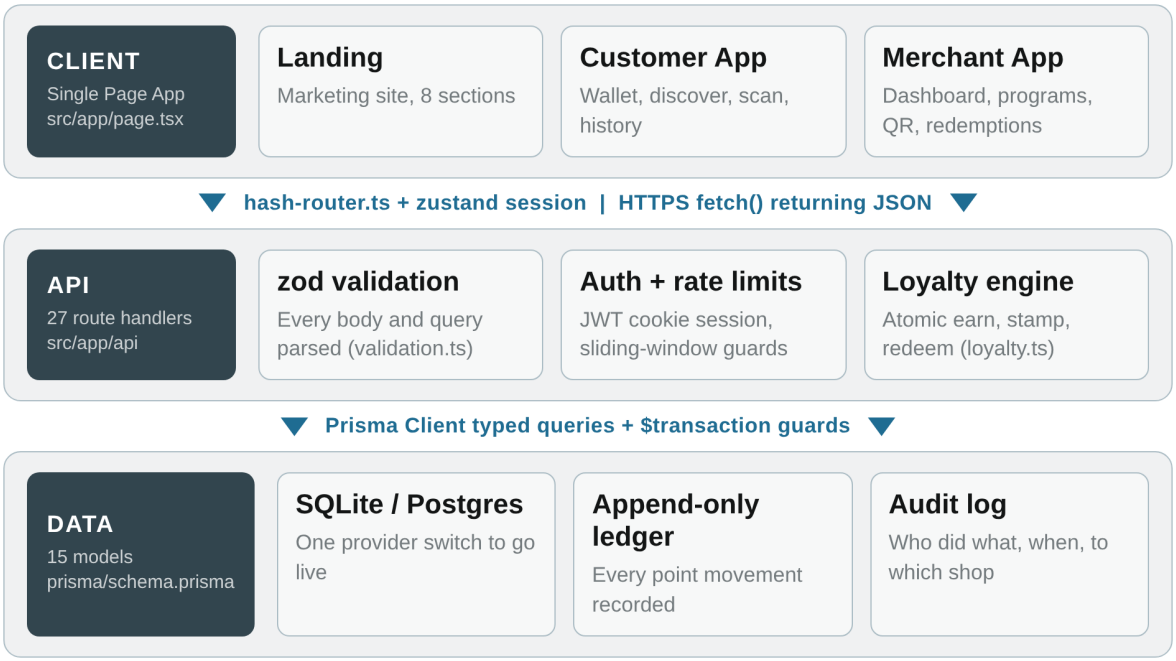


Figure 1 - Three-layer architecture: client SPA, validated API layer, relational data layer.

2.3 Request Lifecycle

1. The SPA calls `fetch()` against a relative /api path with the JWT session cookie attached (httpOnly, SameSite=Lax).
2. The route handler parses the body and query with zod schemas (`validation.ts`); invalid input returns 400 with field errors before any logic runs.
3. Sliding-window rate limits apply per IP and per target (OTP: 5 per 15 minutes, 60-second resend cooldown).

4. Authorization runs: the session JWT identifies the user and role, and every merchant route re-checks BusinessMember ownership of the business being touched.
5. Business logic executes inside the loyalty engine with Prisma $transaction guards; balances are only ever mutated server-side.
6. Every state change writes an append-only LedgerEntry and, where relevant, an AuditLog row and Notification rows.
7. The API answers with a JSON envelope: creations wrap the payload ({business: ...}), errors are {error} with a precise status code.

2.4 Session Model

Property	Value
Token	Signed JWT (jose library), 30-day sliding expiry
Transport	Cookie pl_session - httpOnly, SameSite=Lax, Secure in production
Payload	{ userId, role } where role is CUSTOMER or MERCHANT_OWNER
Who verifies	Every protected route handler + client hydration via GET /api/me
Password model	None - passwordless OTP only, so there are no passwords to leak

Table 2 - Session model summary.

3. Frontend Implementation

The frontend is a TypeScript-first React codebase on Next.js 16 (App Router). All interactive components are client components marked with 'use client'; the app directory also hosts the API route handlers. The UI library is shadcn/ui (New York style) on Radix primitives, styled with Tailwind CSS 4 design tokens defined in globals.css.

Technology	Role	Key locations
Next.js 16 + TypeScript 5	Framework, routing shell, API host	src/app/page.tsx, src/app/api/**
Tailwind CSS 4	Design tokens as CSS variables, utility styling	src/app/globals.css
shadcn/ui + Radix	Buttons, tabs, dialogs, sheets, accordions, OTP input, toasts	src/components/ui/** (45 components)
framer-motion 12	Reveal animations, hero word assembly, parallax, hover lifts	src/components/landing/useReveal.ts
zustand	Client session store (user, role, hydration)	src/lib/store.ts

Technology	Role	Key locations
Custom hash router	Route parsing and programmatic navigation	src/lib/hash-router.ts
qrcode	Renders real scannable QR images in the merchant studio	src/components/merchant/QrStudio.tsx
recharts	Merchant analytics bar chart (real aggregates only)	src/components/merchant/Analytics.tsx
input-otp	Six-slot OTP entry with paste support	src/components/ui/input-otp.tsx
lucide-react	Single icon pack across every surface	all components
Bricolage Grotesque + Manrope	Display and body fonts (next/font, no Inter as display)	src/app/layout.tsx

Table 3 - Frontend stack and where each piece lives.

3.1 Route Map

Hash route	Surface	Implementation
#/	Landing page (8 sections)	src/components/landing/*
#/auth	Login / register, customer or business tabs	src/components/auth/AuthFlow.tsx
#/app	Customer shell with bottom navigation	src/components/views/CustomerApp.tsx
#/app (tabs)	Home, Discover, Scan, History, Profile	src/components/customer/*Tab.tsx
#/business/{id}	Per-business wallet, rewards, feedback	src/components/customer/BusinessDetail.tsx
#/o/{code}	Outlet entry from scanned static QR	src/components/views/OutletEntry.tsx
#/scan/confirm	One-time amount QR confirmation	src/components/views/TxnConfirm.tsx
#/merchant (+7 sub-routes)	Merchant dashboard, programs, QR studio, redemptions, transactions, analytics, settings	src/components/merchant/*
#/privacy, #/terms	Legal pages (DPDP-aware wording)	src/components/views/StaticPage.tsx

Table 4 - Hash route map of the SPA.

3.2 Design System Highlights

- **Palette.** Warm cream canvas #FAF7F2, forest #163300 for headlines and dark sections, lime #9FE870 reserved as the single accent per viewport, marigold #F59E0B for points warmth, sienna for warm alerts. No indigo, no blue defaults.

- **Type.** Bricolage Grotesque for display at 800 weight with negative tracking; Manrope for body; Geist Mono for micro-labels. Numerals in balances are tabular.
- **Shape and texture.** Pill CTAs, 10-28 px card radii with nested-radius math, canvas-tinted layered shadows, grain and dot-grid textures on dark cards, perforated stamp-card language.
- **Motion law.** Failure-safe reveals (content visible without JavaScript, 2.5 s CSS failsafe), one clip-path reveal maximum, press feedback scale(0.96)/150 ms, named transitions only, prefers-reduced-motion honored everywhere.
- **Honesty rules.** Real data only, empty states until real activity exists, DEV MODE banner for unconfigured email/SMS, illustrations labeled as illustrations.

4. Backend Implementation

The backend is 27 Next.js route handlers under src/app/api, all returning JSON. Shared logic lives in focused lib modules so route handlers stay thin: validate, authorize, delegate, respond.

Endpoint group	Routes	Purpose
Auth	POST /api/auth/otp/request, /verify; POST /api/auth/logout; GET /api/me	Passwordless OTP login and registration, session hydration, logout
Public directory	GET /api/businesses; GET /api/businesses/{id}	Discoverable business list and public business profile
Outlets and visits	GET /api/outlets/{code}; POST /api/outlets/{code}/visit	Resolve scanned static QR; record check-in and award stamp
Transaction QR	POST /api/merchant/qr/txn; GET /api/qr/txn/{token}; POST /api/qr/txn/{token}/confirm	Create one-time amount QR (5-min TTL); preview; atomic confirm with award
Wallet	GET /api/wallet; GET /api/wallet/{businessId}; POST /api/wallet/{businessId}/redeem	All accounts, one business wallet, request redemption
Merchant programs	GET/POST/PATCH /api/merchant/program; GET/POST /api/merchant/business	Program builder (draft/active/paused), business profile and outlets
Merchant rewards	GET/POST /api/merchant/rewards (PATCH/DELETE by id)	Reward catalog management per program
Merchant redemptions	GET /api/merchant/redemptions; POST /api/merchant/redemptions/{id}/decide	Queue + approve/reject with atomic decrement
Merchant ops	GET /api/merchant/txns; GET /api/merchant/analytics; POST /api/merchant/outlets	Transaction log, real aggregate analytics, outlet management

Endpoint group	Routes	Purpose
Feedback and notifications	POST /api/feedback; GET /api/notifications; POST /api/notifications/read	1-5 star feedback with 24 h dedupe; in-app notification feed

Table 5 - All 27 endpoints, grouped by concern.

4.1 Lib Modules

Module	Responsibility
validation.ts	30+ zod schemas: OTP, business, program (per-type config), reward, txn, redeem, decide, feedback, queries; parseProgramConfig normalizes string-or-object configs
otp.ts	requestOtp / verifyOtp: codes hashed with salt (sha256), 10-min expiry, max 5 attempts, resend cooldown, user-existence checks before code consumption
session.ts	createSession / destroySession: jose JWT into httpOnly cookie; getSessionUser helper for route guards
delivery.ts	sendOtpEmail / sendOtpSms adapters; reports delivery: 'dev' honestly when SMTP/Resend/Twilio env is missing
rate-limit.ts	In-memory sliding window keyed by IP and target; rateLimitOrThrow raises 429
loyalty.ts	The engine: program resolution, point math, stamp cooldown, atomic issuance, redemption lifecycle (details in 4.2)
db.ts	Prisma client singleton
auth-helpers.ts	requireUser / requireMerchant wrappers returning typed 401/403 responses
api.ts + store.ts	Client side: typed fetch wrapper with cookie handling; zustand session store

Table 6 - Shared library modules.

4.2 Loyalty Engine Functions

Function	Guarantees
assertSingleActiveProgram	A business has at most one active program of a type; duplicates rejected with 409
pointsForAmount	Pure point math from amount in paise and program config (e.g. 10 points per Rs 100); no floating point
confirmTxnAndAward	Validates TxnQr token, expiry (5 min), single-use and outlet binding; updateMany conditional guard makes replay attempts return 409; awards inside one transaction
recordVisitAndStamp	Static-QR visit stamping with server-enforced 30-minute cooldown comparing only STATIC_QR visits, so purchases never block legitimate check-ins

Function	Guarantees
createRedemption	Server-side balance check, PENDING redemption created; balance is untouched until approval
decideRedemption	Approve: atomic decrement + REDEEM ledger entry in one transaction; double-approve returns 409; reject: returns reward to stock if limited

Table 7 - Loyalty engine core functions.

5. Data Model

Fifteen Prisma models model the whole domain. SQLite in the sandbox; one line changes the provider to PostgreSQL for production. The schema deliberately avoids arrays in primitives and keeps every money-adjacent fact in its own table so it can be audited and constrained.

Model	Purpose	Key constraint / rule
User	Identity by Indian mobile or email	role: CUSTOMER | MERCHANT_OWNER
Business	A shop: name, category, city, config	Seeded demo shop is labeled 'Demo Cafe'
Outlet	Physical counter holding the static QR code	Unique outlet code drives /o/{code} links
BusinessMember	Owner-to-shop authorization map	Unique (businessId, userId); every merchant route re-checks it
LoyaltyProgram	Rules engine: STAMPS or POINTS, config JSON	draft/active/paused; zod-validated config; one active per type
LoyaltyAccount	Balance per customer per business	UNIQUE (customerId, businessId) - balances can never mix
LedgerEntry	Append-only money trail	EARN/REDEEM/ADJUST/EXPIRE + running balanceAfter; never updated
Visit	One check-in per scan	source: STATIC_QR | TXN; feeds stamp cooldown
TxnQr	One-time purchase QR	Random 32-char token, amount in paise, 5-min TTL, single-use consumedAt
Reward	What stamps/points can buy	Active flag; cost in points or stamps; stock optional
Redemption	Reward claim lifecycle	PENDING -> APPROVED/REJECTED; approval does the debit; 409 on double approval
Feedback	Star rating + text per business	One per customer per business per 24 h (409 on duplicate)
OtpCode	Login codes	sha256 hash + salt, 10-min expiry, attempts counter

Model	Purpose	Key constraint / rule
AuditLog	Security forensics	actorType/actorId/action/entity/meta on sensitive mutations
Notification	In-app activity feed	Powers customer 'Recent activity' and reward updates

Table 8 - The 15 models and their rules.

UNIQUE (customerId, businessId)

The core invariant: every loyalty balance belongs to exactly one customer at exactly one business.

6. Feature Walkthroughs

6.1 Signup and OTP Login

Authentication is passwordless. A customer or merchant enters an Indian mobile number (validated ^[6-9][0-9]{9}$) or an email; the server creates a hashed, single-purpose OTP and, when no email/SMS provider is configured, returns it with delivery: 'dev' so the UI can show an honest DEV MODE banner instead of pretending a message was sent. Verifying the code either logs the user in or registers them (intent-driven), then sets the session cookie.

Step	Frontend	Backend	Control
Choose role and channel	AuthFlow.tsx tabs (Customer / Business Owner; Mobile / Email)	otpRequestSchema validates target and intent	Rate limit per IP and target
Request code	Send my code button, 60 s resend countdown	POST /api/auth/otp/request creates hashed OtpCode	Max 5 per 15 min; code 6 digits, 10-min expiry
Enter code	input-otp six slots; DEV MODE banner shows the code when delivery is 'dev'	POST /api/auth/otp/verify	sha256(code+salt) compare; max 5 attempts; user-existence check before consuming the code
Session	zustand hydrates via GET /api/me; role routes to #/app or #/merchant	jose JWT cookie issued (30-day)	httpOnly + SameSite=Lax; role checked on every protected route

Step	Frontend	Backend	Control
Merchant onboarding	BusinessRegisterForm reused inside merchant app	POST /api/merchant/business + default outlet	Role gate: MERCHANT_OWNER only

Table 9 - Signup and login flow with controls.

6.2 Static Check-in QR and Stamping

Each outlet prints a static QR from the QR Studio. Scanning opens the outlet entry view; one tap records the visit. The engine awards one stamp if the program is stamp-based and the customer has not already checked in within the 30-minute cooldown. Cooldown comparisons consider only STATIC_QR visits, so a purchase (TXN source) never blocks a legitimate check-in.

Step	Frontend	Backend	Control
Print counter QR	QrStudio.tsx renders perforated print card, PNG download, copy link	QR encodes /o/{outletCode}	Outlet-bound code; no points on this QR, stamps only
Scan and open	OutletEntry.tsx shows business name and program	GET /api/outlets/{code}	Public, read-only
Check in	One tap button	POST /api/outlets/{code}/visit -> recordVisitAndStamp	30-min cooldown; atomic stamp increment + ledger entry; 1 stamp per visit regardless of scan count

Table 10 - Static QR flow.

6.3 One-Time Amount QR (Purchase Points)

For points-per-spend programs the merchant types the bill amount; the server mints a one-time transaction QR that expires in five minutes. The customer scans, sees shop name and amount, and confirms. The engine then validates everything server-side and awards points atomically.

Step	Frontend	Backend	Control
Create	QrStudio amount tab with live 5:00 countdown	POST /api/merchant/qr/txn	Merchant-owned outlet; amount bounds validated; token = 32 random chars
Scan preview	TxnConfirm.tsx shows business, amount, points to earn	GET /api/qr/txn/{token}	Expires after 5 min; single-use token

Step	Frontend	Backend	Control
Confirm	Confirm button, clear success state	POST /api/qr/txn/{token}/confirm -> confirmTxnAndAward	Conditional updateMany guard: replay returns 409; atomic award + ledger; points credited only to this business account

Table 11 - One-time transaction QR flow.

6.4 Redemption Approval Loop

Redemption is intentionally two-phase: the customer requests, the merchant approves. Balance is checked server-side at request time but only debited at approval time, inside one atomic transaction that also writes the REDEEM ledger entry. A second approval attempt returns 409.

6.5 Feedback and Notifications

- Feedback: 1-5 stars plus text per business, deduplicated per 24 hours with a 409; merchants see ratings in their dashboard.
- Notifications: REWARD_EARNED and REDEMPTION_UPDATE events create in-app rows; the customer home feed reads them; POST /api/notifications/read marks them read.
- Analytics: every merchant number is a live aggregate (counts, sums, repeat rate) - the dashboard cannot show a number the ledger does not support.

7. Security Architecture

Perk Loop is a money-adjacent system: points are purchasable value. The security design assumes attackers will screenshot QR codes, replay requests, edit client state, and share accounts. The defense is architectural: the client never decides anything about value - it can only ask.

7.1 Threat Model

Attack	Defense	Verified
Screenshot or photograph a payment QR and reuse it later	TxnQr token is single-use, expires in 5 minutes, bound to the outlet; consumedAt stamp + conditional updateMany guard	E2E: replay returns 409
Replay the confirm request to earn points twice	Same atomic guard: first confirm flips status; every later call fails with 409 before any ledger write	E2E
Inflate or edit points in the browser (devtools, API calls)	Balances only change inside server-side transactions triggered by verified events; no API accepts a 'set balance' or client-computed amount	By construction

Attack	Defense	Verified
Mix or transfer points between businesses	LoyaltyAccount UNIQUE (customerId, businessId); every ledger row is bound to one businessId	E2E: isolation tests
Redeem with insufficient balance, or approve twice	Server-side balance check at request; approval does an atomic guarded decrement; second approval returns 409	E2E: 400/409 cases
Brute-force or phish OTP codes	Codes hashed with salt, 10-min expiry, max 5 attempts, 60 s resend cooldown, 5 per 15 min per IP and target	E2E: code reuse 400
Act on a business they do not own	BusinessMember ownership re-checked inside every merchant route (403 otherwise); role gates on the client are cosmetic only	E2E: cross-merchant 403
Spam visits to farm stamps	30-minute stamp cooldown on STATIC_QR visits; one stamp per visit; velocity limits on OTP and visit endpoints	E2E: cooldown test
Fake analytics or history to mislead merchants	All dashboards read live aggregates from the ledger; no denormalized counters to forge	E2E: real aggregates
Tamper with audit evidence	AuditLog rows are append-only with no update/delete paths in the API	By construction

Table 12 - Threat model and current defenses.

7.2 Why Points Cannot Be Faked

1. Every award path starts from a server-created, single-use artifact: a Visit row (from a scanned outlet code) or a TxnQr confirmation (from a merchant-minted token). There is no endpoint that awards points on user say-so.
2. Amount math happens on the server from program config; the client never multiplies.
3. Mutations run inside Prisma transactions with conditional guards, so race conditions cannot double-credit.
4. The ledger is append-only with a running balance; any inconsistency is detectable by replaying the ledger.
5. Redemptions require merchant approval - even a compromised customer account cannot convert points to goods without a human at the counter approving.
6. AuditLog records who did what for every sensitive mutation, giving forensics after any incident.

7.3 Already Implemented

Control	Detail
Input validation	zod on every body and query, before logic
AuthN / AuthZ	OTP + JWT cookie; per-route role and ownership checks

Control	Detail
Anti-replay	Single-use tokens + conditional updates on TxnQr and Redemption
Rate limiting	In-memory sliding window per IP/target on OTP and sensitive routes
Ledger integrity	Append-only entries with balanceAfter snapshot
Audit trail	AuditLog on sensitive mutations
Honest delivery	DEV MODE banner instead of fake 'message sent' claims
Secrets handling	No secrets in code; OTP salt and session secret read from environment

Table 13 - Controls already in the codebase.

7.4 Production Hardening Checklist

Item	Action	Priority
Strong secrets	Generate SESSION_SECRET and OTP_SALT as 32+ random bytes (openssl rand -base64 32); store in Vercel env vars; rotate quarterly	Must
Transport security	HTTPS enforced by Vercel; add HSTS header; cookies become Secure automatically in production	Must
Durable rate limiting	Swap in-memory limiter for Upstash Redis (free tier) so limits survive restarts and apply across instances	Must
Security headers	Content-Security-Policy, X-Frame-Options DENY, Referrer-Policy, X-Content-Type-Options via next.config headers()	Should
Database backups	Enable Supabase daily backups; test a restore before launch	Must
Dependency hygiene	npm audit in CI; Dependabot or Renovate weekly; pin versions	Should
Monitoring	Sentry free tier for errors; alert on unusual ledger velocity (points per customer per hour)	Should
PII hygiene	Log redaction for phone/email; DPDP 2023 privacy page exists - add account data export and delete request flow	Should
Abuse response	Admin runbook: suspend business, pause program, inspect AuditLog + ledger for an account	Should

Table 14 - Hardening actions before public launch.

8. Production Deployment Checklist

The application is provider-ready: every external service sits behind an adapter with an environment variable. Going live is configuration, not code.

8.1 Environment Variables

Variable	Purpose	How to produce
DATABASE_URL	Postgres connection string (Supabase)	Supabase project settings -> Database
SESSION_SECRET	Signs JWT session cookies	openssl rand -base64 32
OTP_SALT	Salt for hashing OTP codes	openssl rand -base64 32
RESEND_API_KEY	Sends OTP emails	Resend dashboard, after domain verification
TWILIO_* (optional)	Sends OTP SMS for phone login	Twilio console (only if SMS login is wanted at launch)

Table 15 - Required environment variables.

8.2 Steps

1. Create a Supabase project; copy the pooled connection string into DATABASE_URL.
2. Switch the Prisma datasource provider from sqlite to postgresql in prisma/schema.prisma and run prisma db push, then prisma db seed for the labeled demo shop.
3. Create a GitHub repository and push; import the repo in Vercel (framework auto-detected).
4. Add the five environment variables in Vercel project settings (Production + Preview).
5. In Resend, verify the sending domain and add DNS records; set the FROM address for OTP emails.
6. Deploy, then run the E2E smoke suite (scripts/e2e-backend.ts) against the production URL; it exercises 31 checks including replay, cooldown and isolation cases.
7. Register the first real merchant, print the counter QR from the QR Studio, and do one end-to-end stamp and one redemption from a real phone.

8.3 Post-Deploy Smoke Checklist

Check	Expected
OTP email arrives from the verified domain	Within seconds; DEV MODE banner gone
Static QR check-in from two different phones	Second visit within 30 min is cooldown-blocked with a clear message

Check	Expected
Amount QR after 5 minutes	Shows expired; confirm attempt returns 409
Redeem then double-approve	Approve succeeds once; second approve returns 409
Merchant A opens merchant B's redemption	403
Landing, wallet, dashboard on a low-end Android	No layout break; reveal animations respect reduced motion

Table 16 - Manual smoke checklist.

9. Free-Tier Service Selection (Pre-Launch to First 100 Customers)

Strategy for the smallest possible bill: make email the primary OTP channel (the UI already defaults to it), host on free tiers, and keep SMS as an optional paid fallback. On that plan the infrastructure cost for pre-launch and the first 100 customers is effectively zero.

Layer	Pick	Free tier allowance	Why this one
Database	Supabase (Postgres)	500 MB database, 50k monthly active users, daily backups, 2 projects; pauses after 1 week of inactivity	Real Postgres matches the Prisma schema and the atomic-transaction design; Firebase is NoSQL and would force a rewrite of the ledger model
Hosting	Vercel (Hobby)	Free HTTPS, global CDN, 100 GB bandwidth/month	Zero-config for Next.js 16; Cloudflare Pages is cheaper on bandwidth but needs the OpenNext adapter. Note: Hobby license is non-commercial - move to Pro (USD 20/mo) when monetizing
Email OTP	Resend	3,000 emails/month, 100/day	Clean API, adapter already coded in delivery.ts; 100 customers logging in a few times a month fits easily
SMS OTP	MSG91 or Twilio (optional)	MSG91: roughly Rs 0.15-0.25 per India SMS after trial credits; Twilio: small trial credit	Only needed if phone-only customers arrive; email-first keeps this at zero for launch
Rate limit store	In-memory (built-in) then Upstash Redis	Upstash free: 10k commands/day	Single Vercel instance needs none; free Redis tier covers multi-instance later
Error tracking	Sentry	5k errors/month	Free forever tier is enough at this scale

Layer	Pick	Free tier allowance	Why this one
QR rendering	Generated in-browser (qrcode lib)	Unlimited, zero cost	No image service needed at all
Product analytics	Vercel Analytics or Plausible later	Vercel free tier included	Privacy-friendly; upgrade only when funnel questions appear

Table 17 - Recommended services and their free allowances.

INR 0 / month

Estimated infrastructure cost at pre-launch and the first 100 customers (email-only OTP), plus a domain at roughly INR 800/year.

Upgrade triggers to watch: Supabase when the database passes ~400 MB or the project pauses unexpectedly; Vercel Pro at monetization; Resend if OTP volume approaches 3,000/month (about 100 daily active customers logging in); Upstash when the app runs on more than one instance.

10. Optional Later (Phase 2 and Beyond)

Feature	Service / approach	When to add
Google OAuth login	NextAuth v4 (already installed) alongside OTP	When customers ask for one-tap sign-in
Map-based addresses	Google Maps API (USD 200/month credit covers small usage) or MapmyIndia	When onboarding real shops at scale
WhatsApp campaigns	MSG91 or Gupshup WhatsApp API	Phase 2 marketing automation
Tiers and referrals	Schema is ready (LedgerEntry types ADJUST/EXPIRE reserved)	After 100+ active customers
Staff roles and PIN	BusinessMember role extension + staff PIN for counter approvals	When shops have multiple counter staff
SMS-first login for feature-phone users	MSG91 OTP route (adapter exists)	When analytics show demand
Receipt scanning / POS integration	Out of scope until a partner POS is chosen	Much later

Table 18 - Deferred features and their planned services.

Everything above is deliberately out of the current build: the approved plan deferred tiers, referrals, WhatsApp and multi-outlet staff permissions to phase 2, and the schema already reserves room for them.

The current scope - OTP auth, per-business wallets, both QR flows, approval-based redemption, honest analytics - is complete, tested and deployment-ready.

10.1 What Ships Today

Capability	State
Passwordless OTP auth (email + Indian mobile), JWT sessions	Built, 31/31 E2E verified
Per-business wallets with append-only ledger	Built, isolation verified
Static check-in QR with 30-min stamp cooldown	Built, E2E verified
One-time amount QR (5-min TTL, replay-proof)	Built, replay returns 409
Two-phase redemption with merchant approval	Built, double-approve returns 409
Merchant dashboard, program builder, QR studio, analytics	Built, browser-verified
Honest DEV MODE when email/SMS is unconfigured	Built, shown on auth screen
Privacy policy and terms pages (DPDP-aware)	Built, linked in footer and auth

Table 19 - Shipping scope of version 1.0.